

Sabar Mart CRM

How to use, connect and run the customer, vendor and affiliate system

What's inside

- | | | | |
|---|---|----|-------------------------------------|
| 1 | What the CRM does | 7 | Admin and privacy |
| 2 | Getting started: opening and logging in | 8 | Connecting the Node.js store |
| 3 | Who can do what (roles) | 9 | Putting it live on MilesWeb |
| 4 | Support team | 10 | Backups, alerts and troubleshooting |
| 5 | Finance team | 11 | Quick reference |
| 6 | Vendor team | | |

About the pictures

The screenshots are real. They were taken from the CRM itself (version 1.4.0) running with its built-in demo data: sample customers such as Asha Patel and sample vendors such as Kiran Textiles. On your live system you'll see your own data in the same places. The flow pictures are diagrams drawn for this guide.

Version	CRM 1.4.0 · October 2026
For	Support, finance, vendor, affiliate and marketing staff, administrators, and the store developer
Source	github.com/SOLOMAN-DEV/SOLOMAN-DEV · folder <code>sabar-mart-crm</code>

1 · What the CRM does

The Sabar Mart CRM is the one place that keeps track of everything happening around the marketplace: who your customers are and what they buy, how each vendor is performing and what they are owed, which affiliates bring in sales, and which problems need someone's attention.

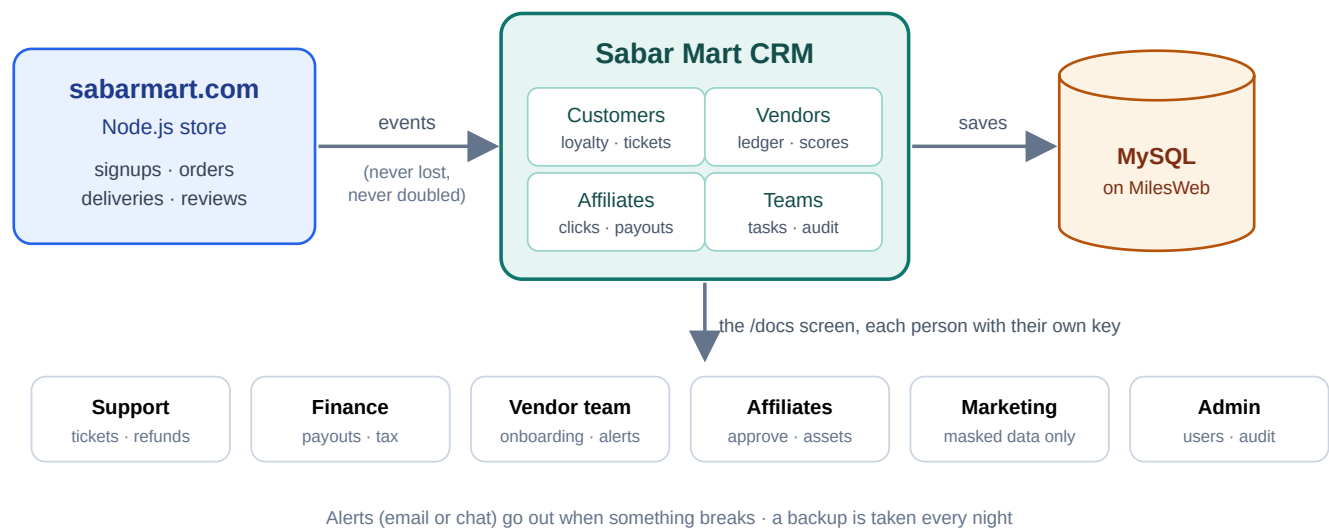


Figure 1. The big picture. The store sends what happens to the CRM. The CRM keeps the records in your MilesWeb database. Each team works in the CRM through its own login.

What it handles for you, automatically

Area	Examples
Customers	Tracks each customer from signup to loyal and VIP, and notices abandoned carts. Awards loyalty points with reward milestones, routes support tickets to the right team, and recommends products (only to customers who agreed to marketing).
Vendors	Document checks and store setup steps, plus scorecards for fulfilment, late shipping, returns and reviews. A full money ledger: every sale, the commission, GST on commission, TCS and TDS, refunds and payouts.
Affiliates	Validates referral clicks, credits orders to the right affiliate, and pays tiered commission only after the 14-day return window. Takes commission back if a paid order is refunded later.
Teams	Raises alerts as tasks, for example a vendor below 3.5★ or a refund over ₹10,000. Gives each person their own key and logs who did what.

The screen you'll use today

Right now staff work in the CRM's built-in API screen, `/docs`, shown throughout this guide. It's a little technical, but every button does exactly one job and each person only sees what their role allows. A friendlier dashboard can be built later on top of the same system.

2 · Getting started

Step 1: open the CRM screen

Open your CRM address followed by `/docs` in Chrome, Edge or Firefox, for example

`https://crm.sabarmart.com/docs` . You'll see the list of everything the CRM can do, grouped by area.

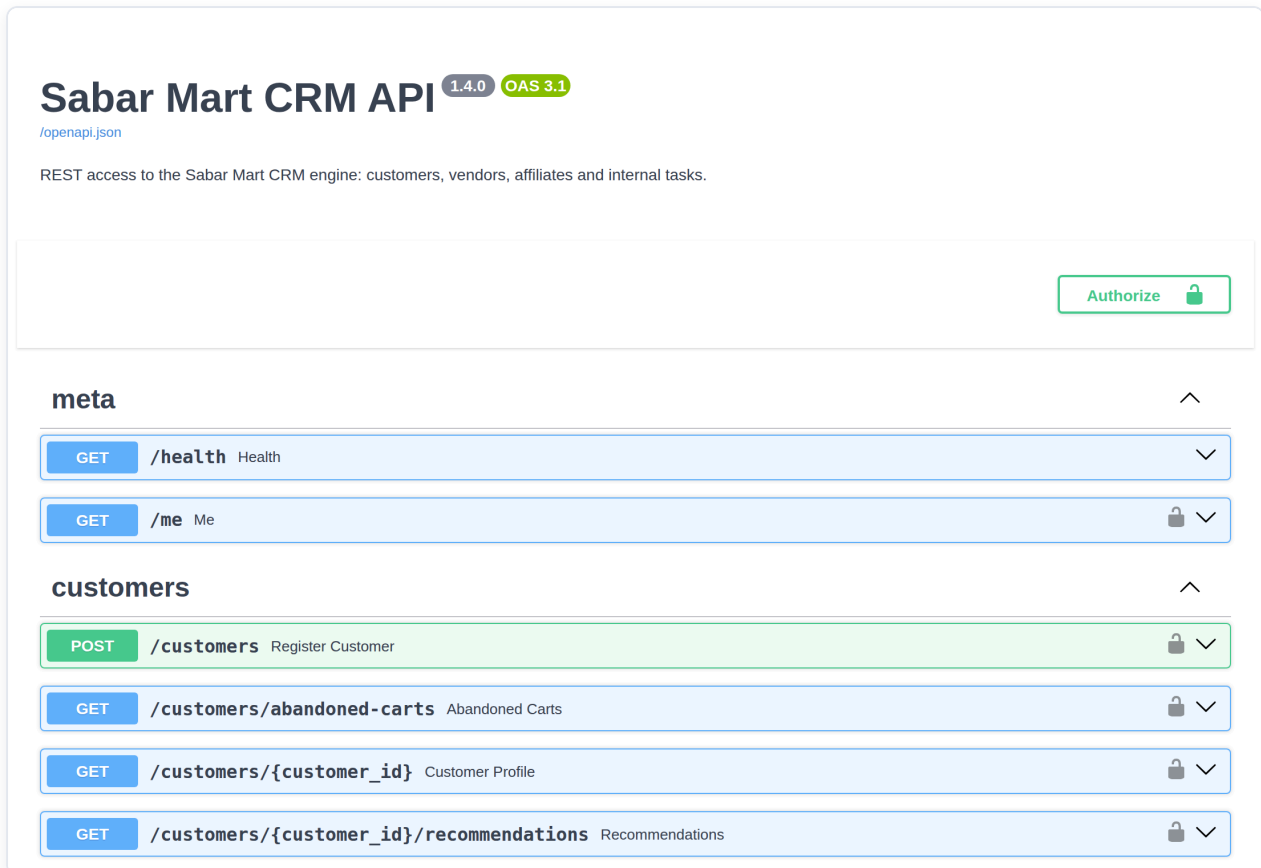


Figure 2. The CRM screen. Each coloured bar is one action. **GET** means "look something up" and changes nothing. **POST** and **PUT** change data. The padlock means you need to be logged in.

Step 2: log in with your key

- 1 Click the green **Authorize** button at the top right.
- 2 Paste your personal key into **Value**. Your administrator gives you this key once; it starts with `smc_`.
- 3 Click **Authorize**, then **Close**. The padlocks now show as closed.

Available authorizations

×

HTTPBearer (http, Bearer)

Value:

demo-support

Authorize

Close

Figure 3. The login box. The demo key `demo-support` is shown here; yours will be a long random key.

Keep your key private

Your key is your identity. Everything done with it is recorded in the audit log under your name. Never share it, never put it in chat or email, and tell an admin straight away if you think someone else has it. They can switch it off in seconds.

Step 3: check who you are logged in as

Open `GET /me`, click **Try it out**, then **Execute**. The answer shows your name, your role and everything you're allowed to do.

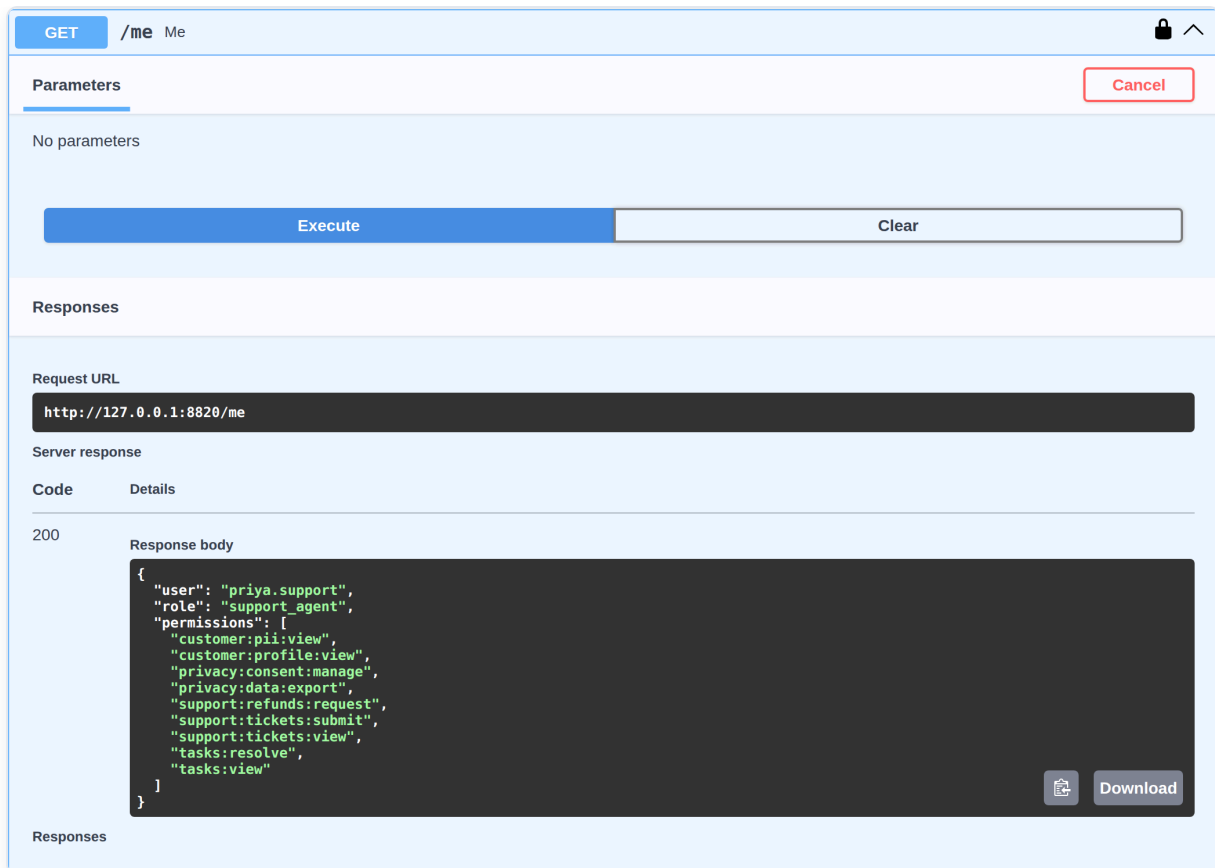


Figure 4. Logged in as `priya.support` with the `support_agent` role.

How every action works: the same five clicks

- 1 Click the bar of the action you need, for example **Customer Profile**, to open it.
- 2 Click **Try it out** on the right.
- 3 Fill in the boxes. Under **Parameters** that means IDs such as `C-1`. Where there's a **Request body**, edit the example text.
- 4 Click the big blue **Execute** button.
- 5 Read the answer under **Server response**: a **code** plus the **response body**.

What the answer codes mean

Code	Meaning	What to do
200 / 201 / 204	Done	Nothing; it worked. 201 means something new was created. 204 means done, with nothing to show.
401	Not logged in	Click Authorize and paste your key again. Your key may have been replaced.
403	Not allowed for your role	This is on purpose (see Figure 5). Ask the right team, or an admin if you think you need access.
404	Not found	Check the ID (customer, order, vendor) for typos.
409	A business rule said no	Read the <code>detail</code> message, e.g. "return window of 14 days has closed".
422	Something in the form is wrong	A box is missing or in the wrong format, e.g. a negative amount.
500	Unexpected error	Send the <code>error_id</code> to your admin. An alert has already gone to them automatically.

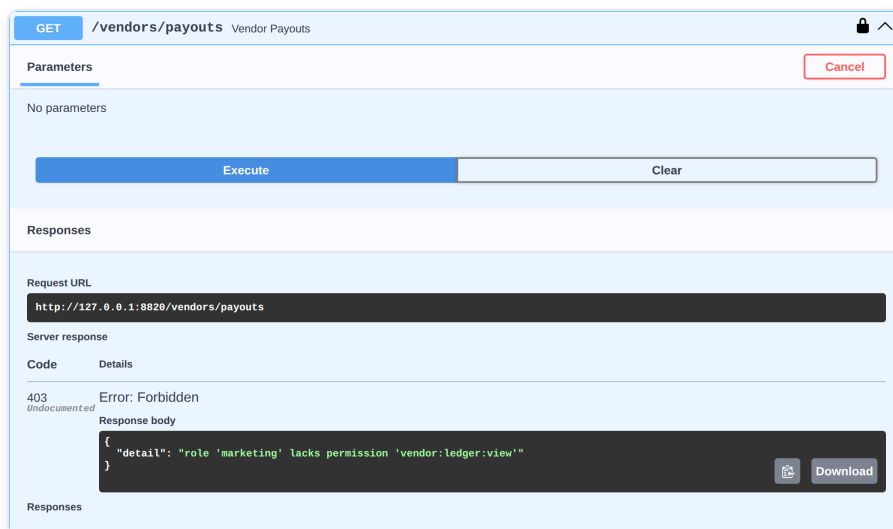


Figure 5. A marketing user trying to open vendor payouts gets **403 Forbidden**: "role 'marketing' lacks permission 'vendor:ledger:view'".

3 · Who can do what

Your key decides your role, and your role decides what you can see and do. Nobody can choose their own role.

Task	Support	Finance	Vendor	Affiliate	Marketing	Admin
See customer profiles	✓	—	—	—	masked	✓
Ticket queue, request refunds	✓	—	—	—	—	✓
Approve large refunds	—	✓	—	—	—	✓
Vendor ledgers, payouts, tax report	—	✓	—	—	—	✓
Vendor onboarding and scorecards	—	view	✓	—	—	✓
Affiliates and marketing assets	—	payouts	—	✓	view	✓
Recommendations (with consent)	—	—	—	—	✓	✓
Customer data export (DPDP)	✓	—	—	—	—	✓
Erase a customer (DPDP)	—	—	—	—	—	✓
Users, audit log, analytics	—	—	—	—	—	✓
Own task queue	support, trust & safety	finance	vendor success	affiliate ops	—	all

There's also a `system` role. It's used only by the store's backend to send events, and it can't read any data.

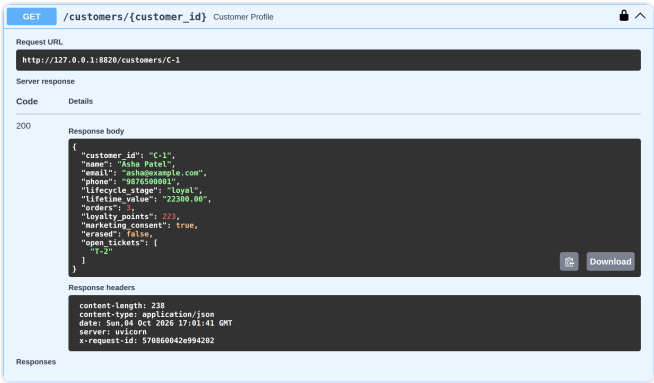


Figure 6a. Support sees Asha's real email and phone number.

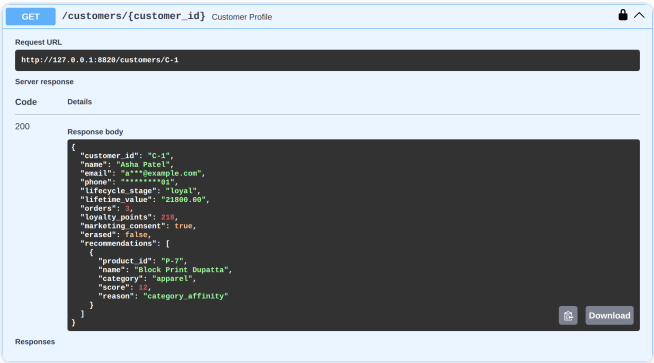


Figure 6b. Marketing sees the same customer with her email and phone hidden: `a***@example.com`, `*****01`.

4 · Support team

Your ticket queue

Open **support** › **GET /tickets**, then **Try it out** › **Execute**. Tickets are already sorted, most urgent first. The CRM reads each message and sets its **priority**, its **team** and the **reasons** for that choice.

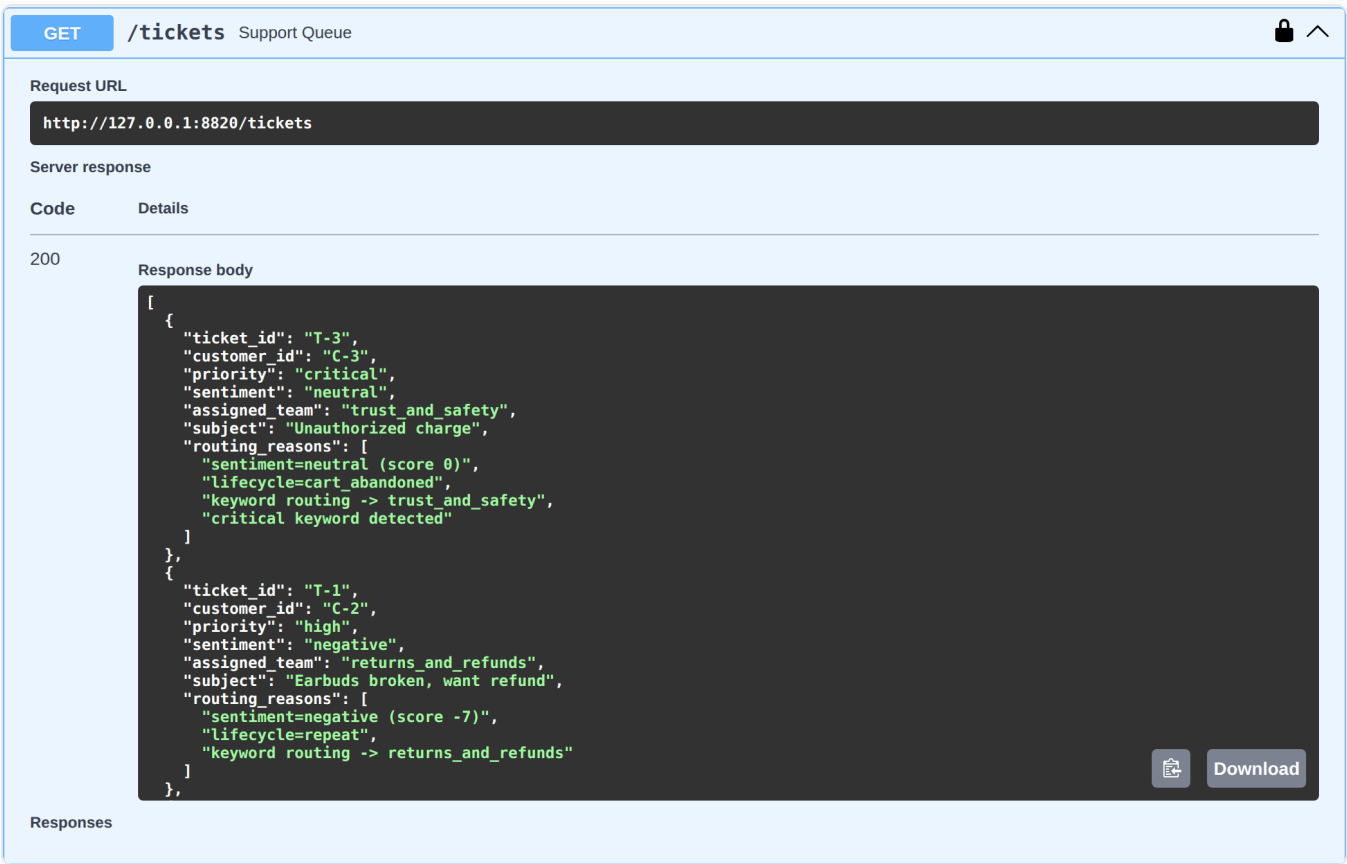


Figure 7. The queue. "Unauthorized charge ... hacked" became **critical** and went to **trust_and_safety**. An angry "broken, want refund" message became **high** and went to **returns_and_refunds**.

The CRM sees...	It does...
fraud, hacked, unauthorized, legal, police	Critical ; goes to trust & safety, and an urgent task is raised automatically
payment, charged, card, UPI	Goes to the payments team
refund, return, exchange	Goes to returns & refunds
delivery, courier, late, missing	Goes to logistics
angry words, a high-value order, a loyal or VIP customer	Priority is raised; VIP general questions go to the VIP concierge

Look up a customer

Open **customers** › **GET /customers/{customer_id}** and enter the customer's ID. You'll see their stage (e.g. *loyal*), lifetime value, loyalty points, marketing consent and open tickets (see Figure 6a).

Give a customer a refund

Open **refunds** › **POST /orders/{order_id}/refunds**, enter the order ID, and in the request body type the **amount** and a short **reason**.

The screenshot shows a REST client interface for a POST request to `/orders/{order_id}/refunds`. The `order_id` parameter is set to `O-1`. The request body is a JSON object: `{ "amount": "500.00", "reason": "Item arrived scratched" }`. The response is a 201 status with a detailed JSON object: `{ "refund_id": "RF-000003", "order_id": "O-1", "customer_id": "C-1", "vendor_id": "V-100", "amount": "500.00", "reason": "Item arrived scratched", "requested_by": "priya.support", "requested_at": "2025-10-04T17:01:43.364219", "status": "approved", "decided_by": "auto", "decided_at": "2025-10-04T17:01:43.364219", "task_id": null }`.

Figure 8. A ₹500 refund on order O-1 for "Item arrived scratched". Because it's under ₹10,000, it was **approved** straight away (`decided_by: auto`). The vendor's balance and the customer's loyalty points were adjusted at once.

Refund rules to know

- **Over ₹10,000:**
the refund waits for finance to approve it. You can't approve your own request.
- **Delivered orders only:**
an order that hasn't been delivered yet should be *cancelled* instead.
- **No more than was paid:**
you can never refund more than the customer paid, and refunds still waiting for approval count against that amount.

A customer asks "what data do you hold about me?"

Open **privacy** › **GET /customers/{customer_id}/export**. The answer contains everything held about them: profile, consent, orders, refunds, tickets and browsing. Click **Download** and send them the file through your normal secure channel. The export is recorded in the audit log.

GET

/customers/{customer_id}/export

Export Customer

🔒 ⬆

Everything held about the customer (DDP right of access).

Request URL

http://127.0.0.1:8820/customers/C-1/export

Server response

Code	Details
200	Response body <pre>{ "subject": { "type": "customer", "id": "C-1" }, "generated_at": "2026-10-04T17:01:44.863856", "erased_at": null, "profile": { "name": "Asha Patel", "email": "asha@example.com", "phone": "9876500001", "signed_up_at": "2026-08-25T17:01:35.343431" }, "consent": { "marketing": true, "updated_at": "2026-08-25T17:01:35.343431", "source": "signup_form" }, "loyalty": { "points": 218, "milestones_awarded": [100] }, "activity": { "browsed_categories": { "apparel": 3 } } }</pre> 📄 Download

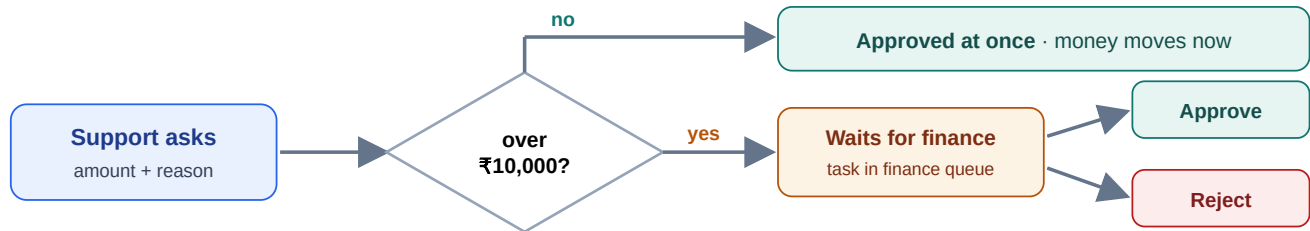
Responses

Figure 9. A data export for customer C-1. Deleting data ("erasure") is for admins only; see section 7.

5 · Finance team

Approve or reject large refunds

Every refund over ₹10,000 appears as a **pending_review** refund and as a task in the **finance** queue.



The person who asked for a refund can never approve it · nothing moves while it waits

Figure 10. How a refund is decided.

- 1 Open **refunds** › **GET /refunds**, type `pending_review` in **status**, and Execute to see what's waiting.
- 2 Check the order and the reason. If it's fine, open **POST /refunds/{refund_id}/approve**, enter the refund ID and Execute. If not, use **/reject** instead.

POST `/refunds/{refund_id}/approve` Approve Refund

Parameters

refund_id * required
string (path)
RF-000002

Execute Clear

Responses

Request URL
`http://127.0.0.1:8820/refunds/RF-000002/approve`

Server response

Code Details
200

Response body

```
{
  "refund_id": "RF-000002",
  "order_id": "O-2",
  "customer_id": "C-1",
  "vendor_id": "V-100",
  "amount": "18000.00",
  "reason": "item damaged in transit",
  "requested_by": "support-demo",
  "requested_at": "2026-10-04T17:01:35.343431",
  "status": "approved",
  "decided_by": "arjun.finance",
  "decided_at": "2026-10-04T17:01:52.258603",
  "task_id": "TSK-00005"
}
```

Responses

Figure 11. An ₹18,000 refund requested by support, approved by `arjun.finance`. Its finance task is closed automatically.

Where the money from a sale goes

Sabar Mart prices **include GST**. When an order is delivered, the CRM records the sale and everything withheld from it. Here is a real example: a ₹1,180 kurta with 18% GST.

Customer pays ₹1,180.00 (GST included)

Vendor receives ₹1,034.76

₹118

Commission ₹118.00 (10% of the price, Sabar Mart's fee)

GST on commission ₹21.24 (18% of ₹118)

TCS ₹5.00 + TDS ₹1.00 (0.5% and 0.1% of the ₹1,000 taxable value)

Taxable value = price ÷ (1 + GST rate) = 1,180 ÷ 1.18 = ₹1,000.00. TCS and TDS are worked out on this, not on the full price.
If the vendor's PAN card isn't verified, TDS is 5% instead of 0.1% (₹50.00 here). Confirm all rates with your CA.

Figure 12. The split of one sale, exactly as the CRM's ledger records it. Each product carries its own GST rate, and an order keeps the rate it was sold at.

Vendor balance and payouts

Open **finance** > **GET /vendors/{vendor_id}/ledger**. **payable_now** is what you can pay today. Money from orders still inside the 14-day return window is held back (**held_in_return_window**), because the customer could still return them.

The screenshot shows a REST client interface with the following details:

- Request URL:** `http://127.0.0.1:8820/vendors/V-100/ledger`
- Server response:** Status 200
- Response body (JSON):**

```
{  "vendor_id": "V-100",  "gross_sales": "22300.00",  "refunds": "18500.00",  "commission_deducted": "380.00",  "gst_on_commission": "68.40",  "tcs_withheld": "16.10",  "tds_withheld": "3.23",  "paid_out": "0.00",  "outstanding_balance": "3332.27",  "held_in_return_window": "1578.44",  "payable_now": "1753.83",  "last_payout_at": null,  "next_payout_due": "2026-10-04T17:01:53.614599"}
```
- Buttons:** A 'Download' button is visible next to the response body.

Figure 13. Ledger for Kiran Textiles: gross sales, refunds, commission, GST on commission, TCS, TDS, the amount held and the amount payable now.

To pay a vendor, use **POST /vendors/{vendor_id}/payouts**. It pays *payable_now*, at most once every 7 days, and only to verified vendors.

Tax report for filing

Open **finance** > **GET /finance/tax-report** and give a **start** and **end** date, e.g. the first day of this month and of next month. For each vendor it shows sales including GST, the taxable value, TCS collected, TDS deducted, and commission with its GST. Use these figures for GSTR-8 and the 194-O return.

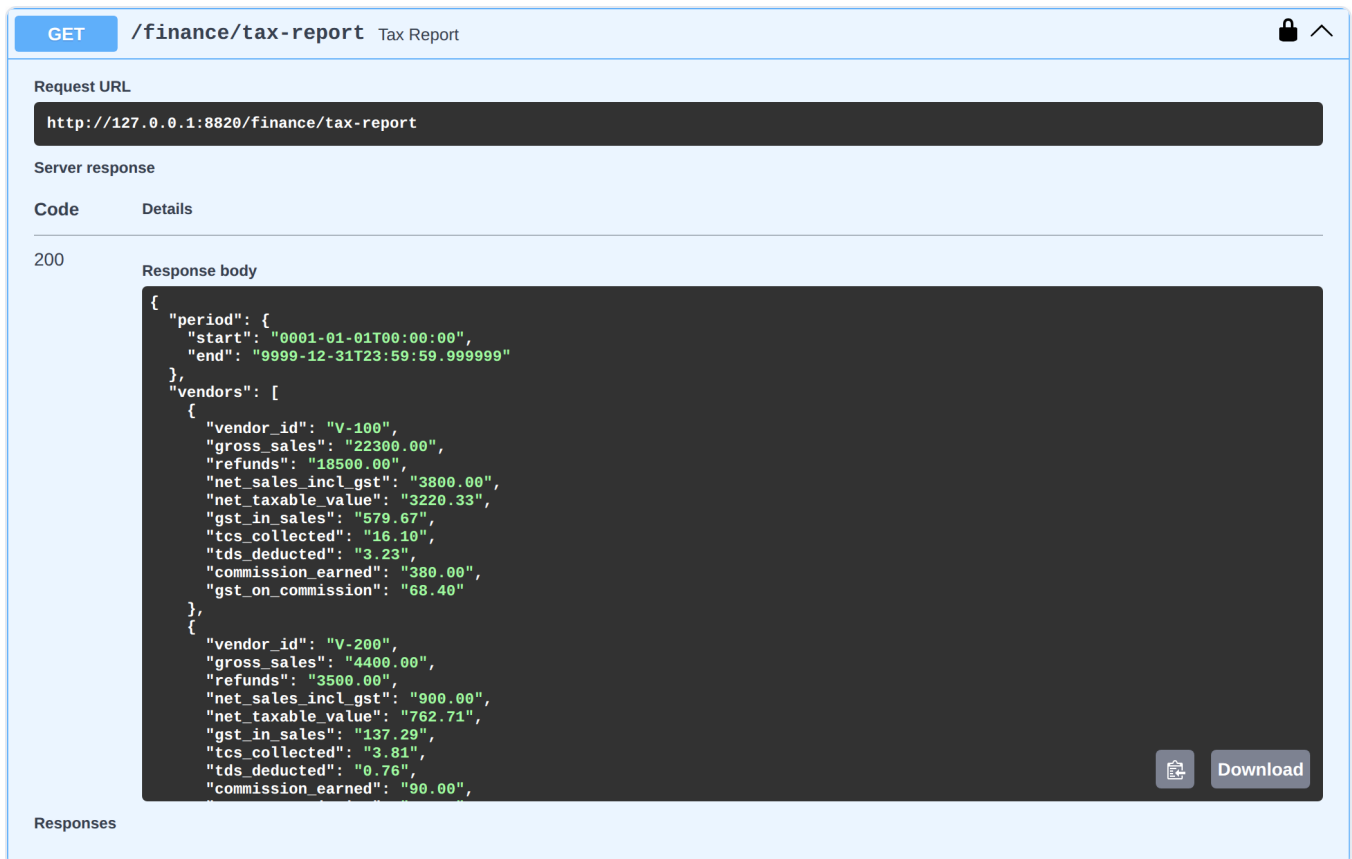


Figure 14. The tax report, one block per vendor. No dates were entered here, so it covers all time. The end of the answer (scroll down in the CRM) also lists the totals and the rates that were used.

Confirm the rates with your Chartered Accountant

The default rates are TCS 0.5%, TDS 0.1% (5% without a PAN) and 18% GST on commission. They follow the law as it stood in late 2024, and they live in one settings file (`config.py`). Have your CA confirm them before go-live, and again after any Budget change. Threshold exemptions are not modelled.

Affiliate payouts

Open **GET /affiliates/{affiliate_id}/payout-preview** to see what a payout would pay right now. If it's right, run **POST /affiliates/{affiliate_id}/payouts**.

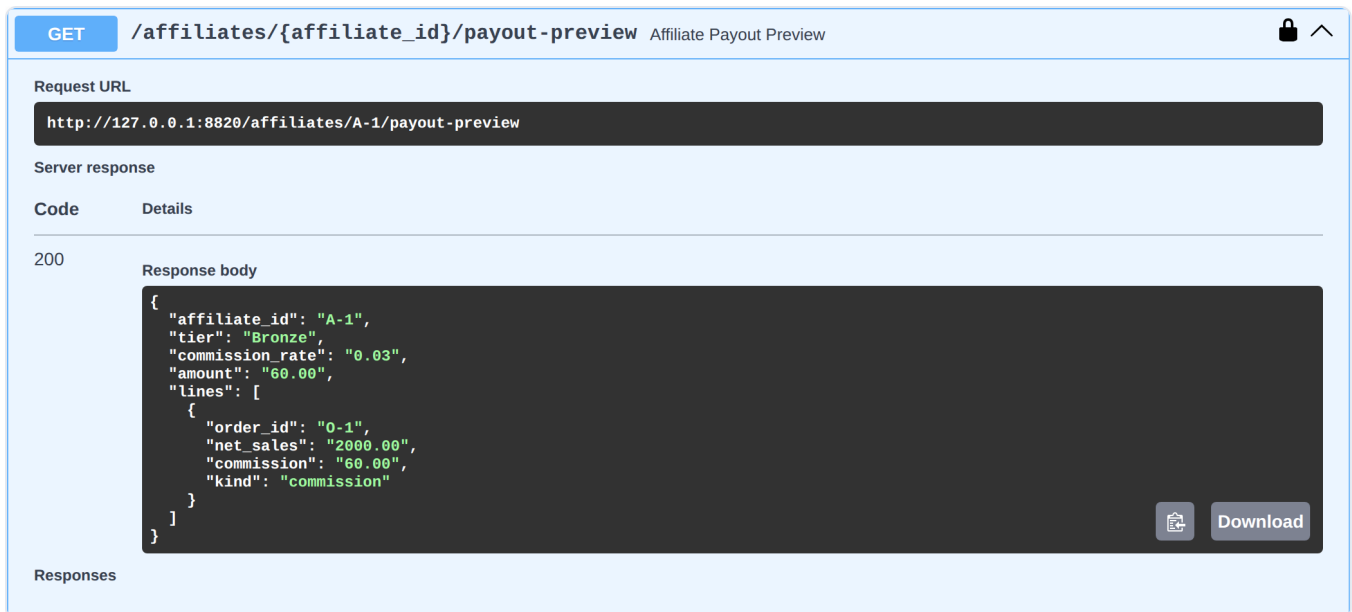


Figure 15. Affiliate A-1 (Bronze tier, 3%) is owed ₹60.00. The referred order was ₹2,500, but support refunded ₹500 on it (Figure 8), so commission is paid on the ₹2,000 that was actually kept. Refunds always reduce affiliate commission automatically.

- **Paid once:** each order's commission is paid only once. Running a payout again pays only what's new.
- **Clawbacks:** if a paid order is refunded later, the next payout automatically deducts the difference.
- **Tiers:** Bronze 3%, Silver 5% (25+ completed orders), Gold 7% (100+) and Platinum 10% (500+). An order's rate is fixed when it's first paid.

6 · Vendor team

Onboarding a new vendor

- 1 **POST /vendors**: create the vendor with an ID, store name and contact email.
- 2 **PUT /vendors/{id}/documents/{doc}**: record each document when it's received. The documents are `business_registration`, `tax_id`, `pan_card`, `bank_details` and `identity_proof`.
- 3 **POST .../documents/{doc}/review**: approve or reject each one, with `{"approved": true}` or `{"approved": false}`.
- 4 **PUT /vendors/{id}/milestones/{m}**: tick off each store setup step: `profile_completed`, `first_product_listed`, `shipping_configured` and `payout_account_linked`.
- 5 **GET /vendors/{id}** shows the progress, what's still missing (`next_actions`) and whether the vendor can go live.

GET /vendors/{vendor_id} Vendor Scorecard

Request URL

http://127.0.0.1:8820/vendors/V-300

Server response

Code	Details
200	Response body <pre>{ "store_name": "Fresh Spices Co", "flagged": false, "vendor_id": "V-300", "total_orders": 0, "fulfillment_rate": null, "shipping_delay_rate": "0", "return_rate": "0", "avg_review_score": null, "review_count": 0, "onboarding": { "vendor_id": "V-300", "store_name": "Fresh Spices Co", "verification_status": "under_review", "documents": { "business_registration": "missing", "tax_id": "pending", "pan_card": "missing", "bank_details": "missing", "identity_proof": "missing" }, "store_setup": { "profile_completed": false, "first_product_listed": false, "shipping_configured": false, "payout_account_linked": false }, "progress_pct": 0, } }</pre>

Responses

Figure 16. "Fresh Spices Co" is `under_review`: its tax ID is pending and four documents are missing, so `can_go_live` is false. The next actions tell you what to chase.

Your alerts: the `vendor_success` queue

Open **internal > GET /tasks** with team `vendor_success`. The CRM raises these tasks by itself. The same problem never creates a duplicate task; the existing task is updated instead. When you've dealt with one, close it with **POST /tasks/{task_id}/resolve**.

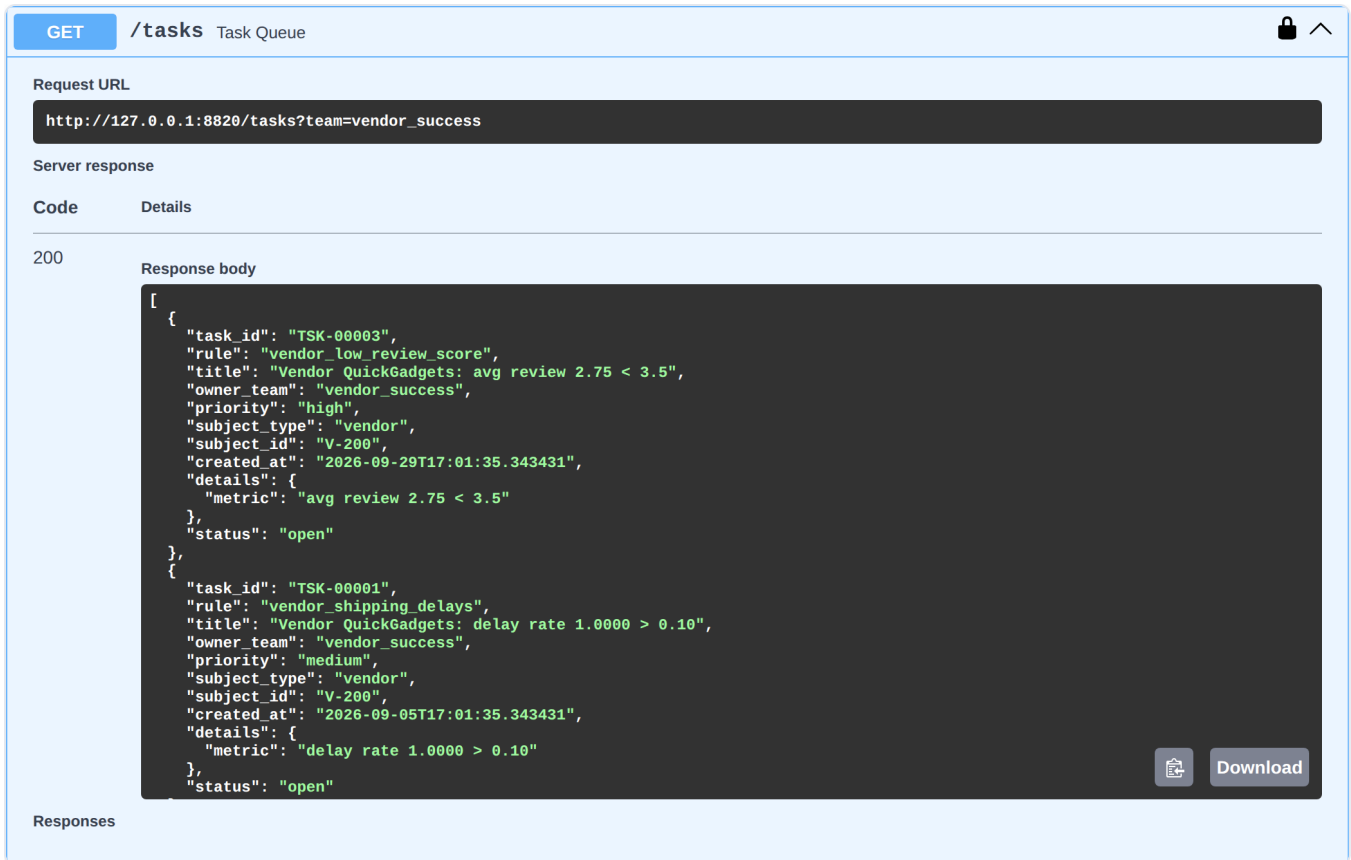


Figure 17. QuickGadgets was flagged for an average review below 3.5★, late shipping and too many returns.

Alert	When	Priority
Low review score	Average review below 3.5★	High
Low fulfilment	Fewer than 95% of orders fulfilled (too many cancellations)	High
Shipping delays	More than 10% of shipments late	Medium
High returns	More than 15% of delivered orders returned	Medium

While a vendor is flagged, their products are left out of customer recommendations.

7 · Admin and privacy

Give a new team member access

Open **admin > POST /users** and enter a username and a role. The answer contains their **API key**. It's shown only this once. Pass it to the person securely; the CRM keeps only a scrambled copy.

POST /users Create User

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8820/users' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer demo-admin' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "neha.support",
    "role": "support_agent"
  }'
```

Request URL

```
http://127.0.0.1:8820/users
```

Server response

Code

Details

201

Response body

```
{
  "username": "neha.support",
  "role": "support_agent",
  "active": true,
  "created_at": "2026-10-04T17:02:02.399322",
  "api_key": "smc_rwM60_K2ChU1lJfLWTPTZRAnogMfsDGB1anruNI_2-E",
  "note": "store this key now; it cannot be shown again"
}
```

Responses

Figure 18. Creating `neha.support` with the `support_agent` role. The key in the answer is what she pastes into Authorize.

When...	Do this
Someone lost their key, or it may be exposed	POST /users/{name}/rotate-key . The old key stops working at once.
Someone leaves the company	POST /users/{name}/deactivate , the same day
You want to see who has access	GET /users

Audit log: who did what

GET /audit lists the latest actions. Filter by person with `actor`. It records every change, every refused attempt, and every look at sensitive data such as customer details, ledgers, payouts and tax reports.

GET

/audit

Audit Log

Request URL

http://127.0.0.1:8820/audit?limit=6

Server response

Code

Details

200

Response body

```
[
  {
    "audit_id": "AUD-0000000009",
    "at": "2026-10-04T17:02:02.399452",
    "actor": "admin.rao",
    "role": "admin",
    "method": "POST",
    "path": "/users",
    "outcome": "success"
  },
  {
    "audit_id": "AUD-0000000008",
    "at": "2026-10-04T17:01:54.873208",
    "actor": "arjun.finance",
    "role": "finance",
    "method": "GET",
    "path": "/finance/tax-report",
    "outcome": "success"
  },
  {
    "audit_id": "AUD-0000000007",
    "at": "2026-10-04T17:01:53.614795",
    "actor": "arjun.finance",
    "role": "finance",
    "method": "GET",
    "path": "/vendors/V-100/ledger",
    "outcome": "success"
  }
]
```

Responses

Download

Figure 19. The most recent audit entries: the time, the person, their role, what they did and whether it worked.

Business overview

GET /analytics gives the whole picture: customers by stage, abandoned carts and marketing opt-ins; orders by status; revenue net of refunds; commission and taxes withheld; flagged vendors; affiliate sales; open tickets; and open tasks.

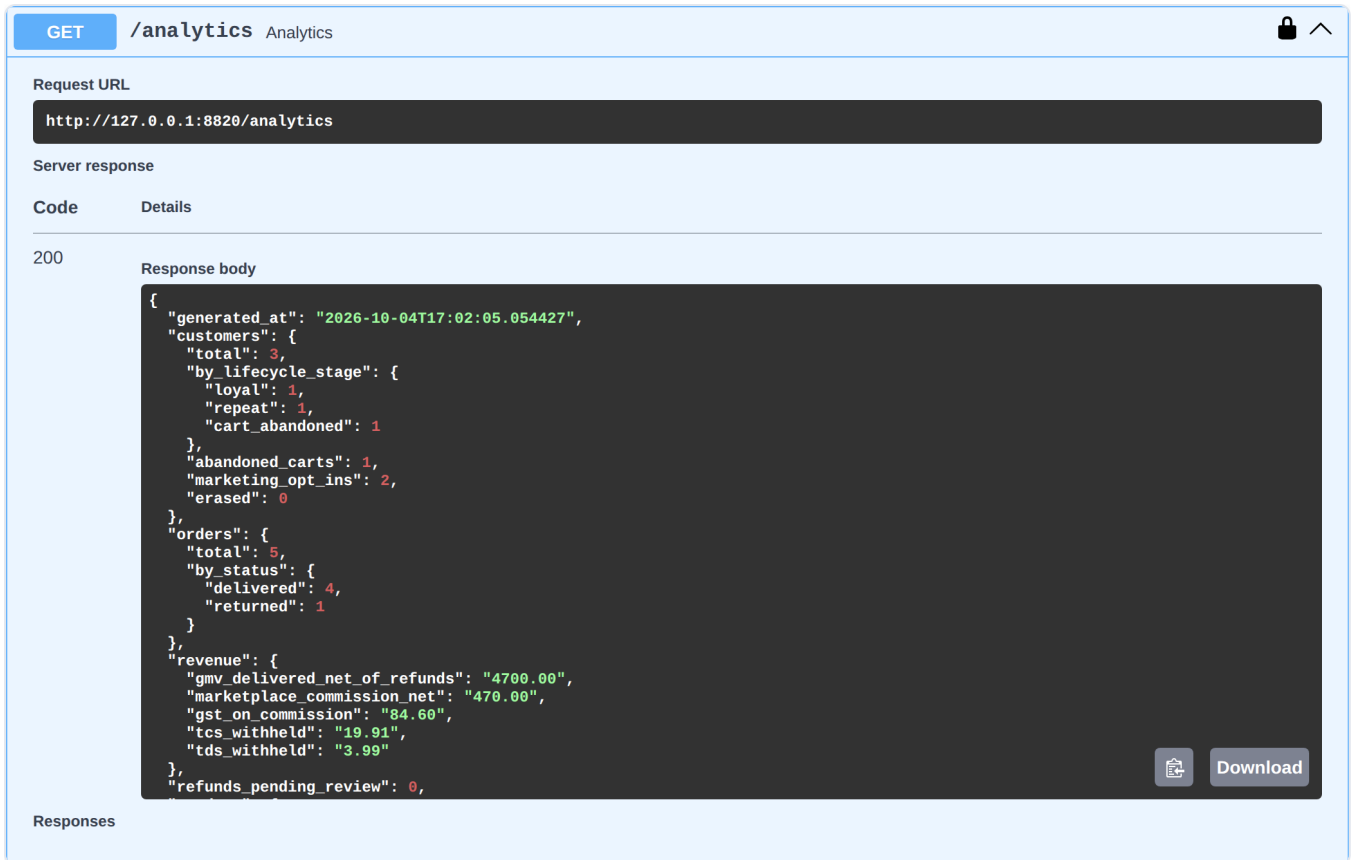


Figure 20. The analytics overview for the demo marketplace.

Privacy requests (DPDP Act 2023)

Request	How	Who
"Stop sending me offers"	POST /customers/{id}/consent with <code>{"marketing": false, "source": "phone_request"}</code> . The store's unsubscribe link does the same automatically.	Support
"What do you hold about me?"	GET /customers/{id}/export (Figure 9)	Support
"Delete my data"	POST /customers/{id}/erase . The name, email, phone, browsing, ticket text and refund reasons are wiped. Orders and the money ledger are kept without the name , because tax law requires those records.	Admin only

Erasure can't be undone

The CRM refuses to erase someone who still has orders on the way or refunds waiting for a decision, or an affiliate who is still owed commission. Finish those first. After an erasure the store can't add new activity for that customer. Older backups still contain the data until they are deleted on schedule.

Outside the CRM you also need a published privacy notice, a grievance officer, and to answer requests within the timelines you promise.

8 · Connecting the Node.js store

For the store developer.

The store sends the CRM a short message, an **event**, every time something happens: a signup, an order, a delivery and so on. The package in `sabar-mart-crm/clients/node` does the hard parts. If the CRM is down, checkout keeps working and nothing is lost, and nothing is ever counted twice.

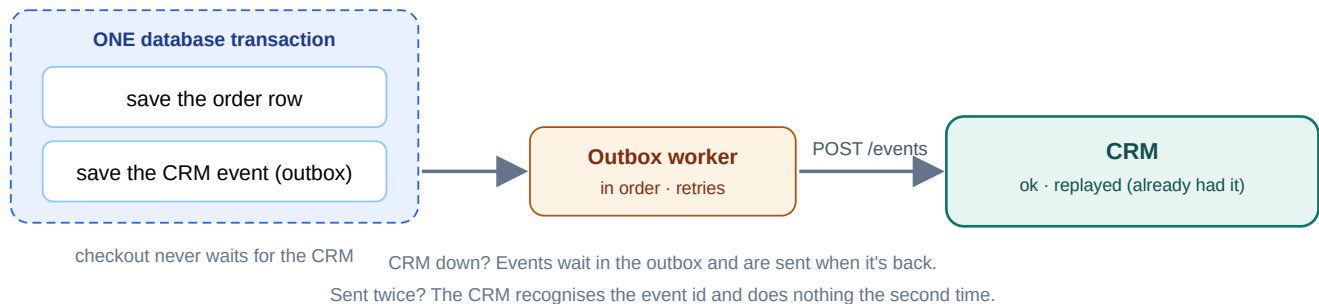


Figure 21. The "outbox" pattern used by the Node.js package. The event is saved with the order, then a background worker delivers it.

Setup, in three steps

- 1 Install the package and `mysql2`, and create the outbox table in the **store's** database from `clients/node/sql/outbox.mysql.sql`.
- 2 Ask the CRM admin for a key with the `system` role.
- 3 Start the worker once when the app boots, and add events inside your existing database transactions:

```
const { CrmClient, MysqlOutboxStore, OutboxWorker, events } = require('@sabarmart/crm-client');

const crm    = new CrmClient({ baseUrl: process.env.CRM_URL, apiKey: process.env.CRM_API_KEY
});
const outbox = new MysqlOutboxStore(pool);           // your mysql2/promise pool
new OutboxWorker({ client: crm, store: outbox }).start(); // safe in every Node process

// inside the checkout transaction (conn = that transaction's connection):
await conn.query('INSERT INTO order_lines ...', [...]);
await outbox.enqueue([events.orderPlaced({
  orderId: 'L-1001', customerId: 'C-1', productId: 'P-1', amount: '1180.00', // GST included
}), conn]);
```

Send this event...	...when
<code>customerRegistered</code>	An account is created. Set <code>marketingConsent: true</code> only if they ticked the opt-in box.
<code>productUpserted</code>	A product is added or changed. Always include <code>gstRate</code> , e.g. "0.18".
<code>orderPlaced</code> / <code>orderShipped</code> / <code>orderDelivered</code> / <code>orderReturned</code> / <code>orderCancelled</code>	For each order line, as it moves through delivery
<code>customerBrowsed</code> , <code>cartItemAdded</code> , <code>cartItemRemoved</code>	Shopping activity
<code>referralClicked</code>	A product page is opened with <code>?ref=CODE</code>
<code>vendorReviewed</code> , <code>ticketCreated</code> , <code>customerConsentUpdated</code>	Reviews, support messages, marketing opt-in or opt-out

A full Express example with every hook point is in `clients/node/examples/express-integration.js` . Vendors must already exist in the CRM before their products arrive. The vendor team creates them (section 6).

9 · Putting it live on MilesWeb

The CRM runs on MilesWeb cPanel hosting using **Setup Python App** and a **MySQL database**. The full step-by-step guide, with exact commands, is in `sabar-mart-crm/DEPLOY_MILESWEB.md`. Here is the overview.

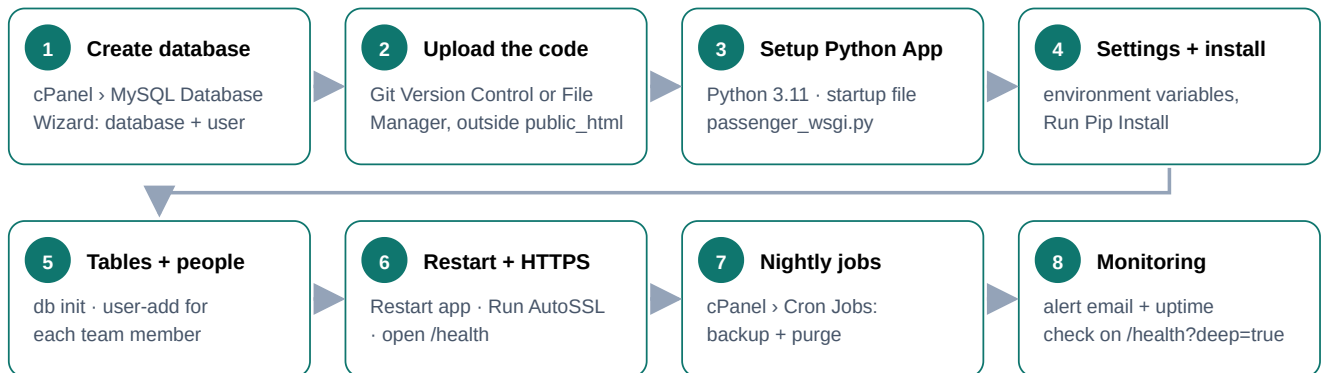


Figure 22. The eight deployment steps (a diagram, not a cPanel screenshot). If *Setup Python App* is missing in your cPanel, ask MilesWeb support to enable it or move your account.

The settings (environment variables)

Name	What to put
<code>DATABASE_URL</code>	<code>mysql+pymysql://USER:PASSWORD@localhost/DBNAME?charset=utf8mb4</code> . URL-encode special characters in the password.
<code>SABAR_CRM_API_KEYS</code>	The store's key: <code>LONGRANDOMKEY:system:storefront</code> . People get personal keys with <code>user-add</code> instead.
<code>SABAR_CRM_ALERT_EMAIL</code> + <code>SABAR_CRM_SMTP_*</code>	Who gets error alerts, and a cPanel mailbox to send them from
<code>SABAR_CRM_DOCS</code>	Leave unset (on) while staff use the /docs screen. The page itself shows no data without a key.

Create the first admin and the team

```
python -m sabar_mart_crm.db init # creates the tables
python -m sabar_mart_crm.db user-add yourname admin # prints your key, shown once
python -m sabar_mart_crm.db user-add priya finance
python -m sabar_mart_crm.db user-add rahul support_agent
```

10 · Backups, alerts and troubleshooting

Nightly backups

In **cPanel** › **Cron Jobs** add two daily jobs. Settings are read from a private file, `~/.sabar-crm.env` (chmod 600); see `DEPLOY_MILESWEB.md` step 7.

When	Command
Daily 02:30	<code>/home/cpuser/sabar-mart-crm/scripts/cron.sh backup</code>
Daily 03:15	<code>/home/cpuser/sabar-mart-crm/scripts/cron.sh purge</code>

- **What a backup is:** a single compressed file holding the whole CRM, readable only by your account. It's checked after it's written. The newest 14 are kept.
- **If one fails:** an alert email is sent.
- **Copy them off the server** every week. A backup on the same server doesn't help if the account is lost.
- **To restore** into an empty database: `python -m sabar_mart_crm.db restore FILE`.

Alerts and health checks

- **Crashes:** if something crashes, the admin gets an email with an **error id**. The person who hit the error sees the same id, so you can match them up. The same problem alerts at most once every 15 minutes.
- **Uptime monitor:** point a free monitor such as UptimeRobot at `https://crm.sabarmart.com/health?deep=true`. It answers 200 when everything is up, and 503 when the database can't be reached.
- **Store sync:** the store developer should watch `outbox.stats()`. `pending` should stay near 0, and `dead` should be 0.

Troubleshooting

Problem	Likely cause and fix
401 on every action	Not logged in, or your key was replaced or switched off. Click Authorize again, or ask an admin for a new key.
403 on an action you need	Your role doesn't include it (section 3). Ask the right team, or ask an admin to review your role.
409 "return window ... has closed"	Returns are only allowed within 14 days of delivery. Use a goodwill refund instead.
409 "exceeds refundable"	The amount is more than what's left to refund, including refunds still waiting for approval.
409 "customer ... has been erased"	That customer was deleted on request (DPDP). No new activity can be added.
503 "timed out waiting for the CRM database lock"	The server is overloaded or a request got stuck. Try again in a minute. If it keeps happening, the admin will have an alert.
A new order doesn't appear in the CRM	Check the store's outbox: is the event <code>pending</code> (the CRM was unreachable) or <code>dead</code> (rejected: read <code>last_error</code>)? A common cause is that the vendor or product doesn't exist in the CRM yet.
The app won't start on MilesWeb	Read <code>stderr.log</code> in the app folder. Check <code>DATABASE_URL</code> , run <i>Run Pip Install</i> again, and check the Python version is 3.10 or newer.

11 · Quick reference

I want to...	Action on the /docs screen	Role
See my role and permissions	GET /me	anyone
See the support queue	GET /tickets	support
Look up a customer	GET /customers/{customer_id}	support, marketing
Refund a customer	POST /orders/{order_id}/refunds	support
See refunds waiting for approval	GET /refunds?status=pending_review	finance
Approve / reject a refund	POST /refunds/{refund_id}/approve · /reject	finance
See what a vendor is owed	GET /vendors/{vendor_id}/ledger	finance
Pay a vendor	POST /vendors/{vendor_id}/payouts	finance
Get the tax report	GET /finance/tax-report?start=...&end=...	finance
Pay an affiliate	GET .../payout-preview, then POST /affiliates/{id}/payouts	finance
Onboard a vendor	POST /vendors, then documents and milestones	vendor
Check a vendor's progress or score	GET /vendors/{vendor_id}	vendor, finance
See my team's alerts	GET /tasks?team=...	each team
Close an alert	POST /tasks/{task_id}/resolve	each team
Approve an affiliate	POST /affiliates/{affiliate_id}/approve	affiliate
Record a marketing opt-out	POST /customers/{id}/consent	support
Export a customer's data	GET /customers/{id}/export	support
Erase a customer	POST /customers/{id}/erase	admin
Add a team member / replace a key	POST /users · /users/{name}/rotate-key	admin
Remove a leaver's access	POST /users/{name}/deactivate	admin
See who did what	GET /audit	admin
See the business overview	GET /analytics	admin

Key numbers

Return window	14 days after delivery
Refunds needing finance approval	over ₹10,000
Vendor review alert	below 3.5★
Commission / GST on commission	10% of price / 18% of commission
TCS / TDS (confirm with your CA)	0.5% / 0.1% of taxable value (5% TDS without a PAN)
Vendor payout cycle	at most once every 7 days
Affiliate tiers	Bronze 3% · Silver 5% · Gold 7% · Platinum 10%
Backups kept	newest 14, taken nightly

All these numbers live in one settings file (`sabar_mart_crm/config.py`) and can be changed without touching anything else.